

CM06 - Pointers

Pointer definition, dereference, pointer-to-pointer, arithmetic

Louis Ledoux

2026-07-05

Pointer Type and Use

Pointer Type and Use

Live pacing: about 8 min

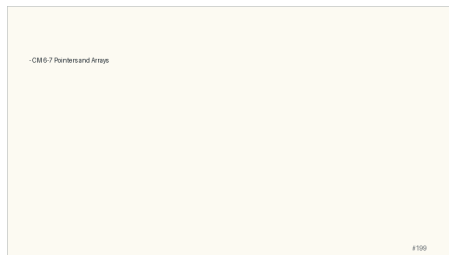
Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

Main Path

- ▶ Start from the question: What does a pointer value name?
- ▶ Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- ▶ End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- ▶ Representative source slide: 200
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Pointer type

- Syntax
- type `*` identifier
- Function
- The pointer type, `*`, is applied over a specific type of an object
- The type can be anything: integer, character, float, pointer, structure, function etc.
- The identifier points (refers) to an object of that type
- It contains the address of the object that it is pointing to

#200

Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Pointer

- Useful
- Mechanism of indexing
- List structures

#201

Backup Index

Original slides 199-201

-CM 6-7 Pointers and Arrays

Backup: CM 6-7 Pointers and Arrays

Original slide 199

-CM 6-7 Pointers and Arrays

Backup: Pointer type

Original slide 200

- ▶ Syntax:
- ▶ `type * identifier;`
- ▶ Function:
- ▶ The pointer type, `*`, is applied over a specific type of an object
- ▶ The type can be anything: integer, character, float, pointer, structure, function etc.
- ▶ The identifier points (refers) to an object of that type
- ▶ It contains the address of the object that it is pointing to

Pointer type

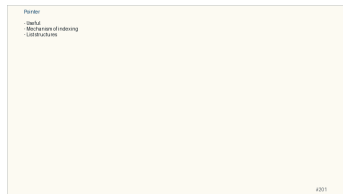
- Syntax:
- `type * identifier;`
- Function:
- The pointer type, `*`, is applied over a specific type of an object
- The type can be anything: integer, character, float, pointer, structure, function etc.
- The identifier points (refers) to an object of that type
- It contains the address of the object that it is pointing to

4200

Backup: Pointer

Original slide 201

- ▶ Useful
- ▶ Mechanism of indexing
- ▶ List structures
- ▶ Passing arguments to a function
- ▶ Dynamic allocation
- ▶ Often associated with NULL
- ▶ NULL is a value of pointer that points to nothing (nothing is an invalid address)
- ▶ Attention CAPITAL letters!!



Pointer Definition and Operations

Pointer Definition and Operations

Live pacing: about 8 min

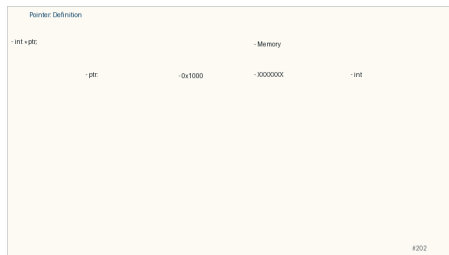
Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

Main Path

- ▶ Start from the question: What does a pointer value name?
- ▶ Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- ▶ End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- ▶ Representative source slide: 203
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Pointer Definition

```
int *ptr;
int x = 10;
int y = 20;
char c = 'X';
```

	- ptr	- 0x1000	- 00000000	- int
- x	- 0x100C	- 10		
- y	- 0x1010	- 20		
- c	- 0x101B	- 65		

- The object `ptr` is declared as a pointer to integers (`int*`)
- It can contain the address of `x` or `y` (both are `int`), but not `c` (as it is a `char`)

#203

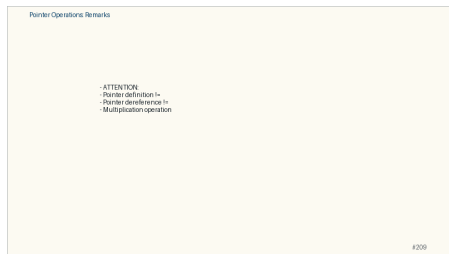
Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 202-209

Pointer: Definition

- int *ptr;

- Memory

- ptr:

- 0x1000

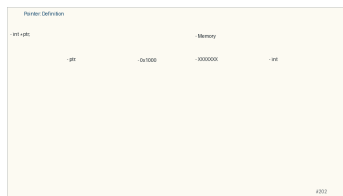
- XXXXXXX

- int

Backup: Pointer: Definition

Original slide 202

- ▶ `int *ptr;`
- ▶ Memory
- ▶ `ptr:`
- ▶ `XXXXXXXX`
- ▶ `int`
- ▶ `0x1000`

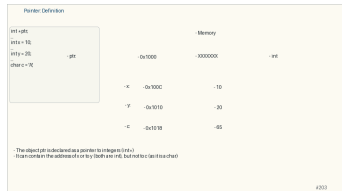


Backup: Pointer: Definition

Original slide 203

```
int *ptr;  
...  
int x = 10;  
...  
int y = 20;  
...  
char c = 'A';
```

- ▶ Memory
- ▶ ptr:
- ▶ XXXXXXXX
- ▶ int
- ▶ 0x1000
- ▶ x:
- ▶ 0x100C
- ▶ 10



Backup: Pointer Operations: Address reception

Original slide 204

```
int *ptr;  
...  
int x = 10;  
...  
int y = 20;  
...  
char c = 'A';  
ptr = &x;
```

- ▶ Memory
- ▶ ptr:
- ▶ XXXXXXXX
- ▶ int
- ▶ 0x1000
- ▶ x:
- ▶ 0x100C

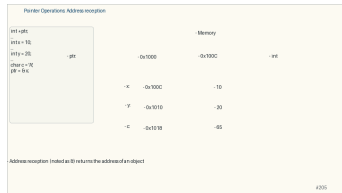


Backup: Pointer Operations: Address reception

Original slide 205

```
int *ptr;  
...  
int x = 10;  
...  
int y = 20;  
...  
char c = 'A';  
ptr = &x;
```

- ▶ Memory
- ▶ ptr:
- ▶ 0x100C
- ▶ int
- ▶ 0x1000
- ▶ x:
- ▶ 0x100C

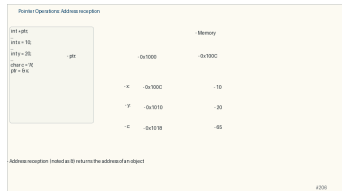


Backup: Pointer Operations: Address reception

Original slide 206

```
int *ptr;  
...  
int x = 10;  
...  
int y = 20;  
...  
char c = 'A';  
ptr = &x;
```

- ▶ Memory
- ▶ ptr:
- ▶ 0x100C
- ▶ 0x1000
- ▶ x:
- ▶ 0x100C
- ▶ 10

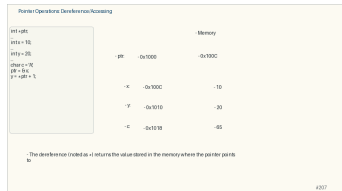


Backup: Pointer Operations: Dereference/Accessing

Original slide 207

```
int *ptr;  
  
...  
int x = 10;  
  
...  
int y = 20;  
  
...  
char c = 'A';  
ptr = &x;  
y = *ptr + 1;
```

- ▶ Memory
- ▶ ptr:
- ▶ 0x100C
- ▶ 0x1000
- ▶ x:
- ▶ 0x100C



Backup: Pointer Operations: Dereference/Accessing

Original slide 208

```
int *ptr;  
  
...  
int x = 10;  
  
...  
int y = 20;  
  
...  
char c = 'A';  
ptr = &x;  
y = *ptr + 1;
```

- ▶ Memory
- ▶ ptr:
- ▶ 0x100C
- ▶ 0x1000
- ▶ x:
- ▶ 0x100C

Pointer Operations: Dereference/Accessing		
<pre>int *ptr; int x = 10; int y = 20; char c = 'A'; ptr = &x; y = *ptr + 1;</pre>		Memory
	ptr	0x1000 0x100C
	*ptr	0x100C 10
	*ptr	0x1010 11
	*ptr	0x1018 85

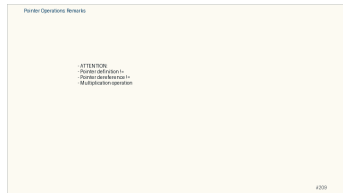
- The dereference (noted as *i) returns the value stored in the memory where the pointer points to.

#208

Backup: Pointer Operations: Remarks

Original slide 209

- ▶ ATTENTION:
- ▶ Pointer definition !=
- ▶ Pointer dereference !=
- ▶ Multiplication operation



Pointer to Pointer

Pointer to Pointer

Live pacing: about 8 min

Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

Main Path

- ▶ Start from the question: What does a pointer value name?
- ▶ Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- ▶ End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Pointer to pointer

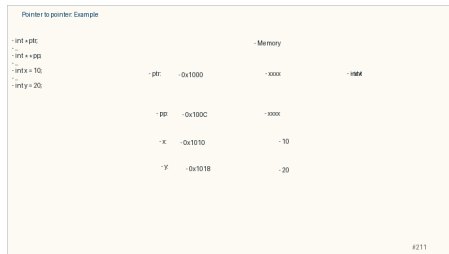
- The object where a pointer points to can be also a pointer
- In this case, the type of the object is type `*`, i.e. pointer type
- `int **pp` means that `pp` is a pointer that points to a pointer that points to integers

- type
- type
- int - * - ** - pp

#210

Worked Example

- ▶ Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- ▶ Representative source slide: 211
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



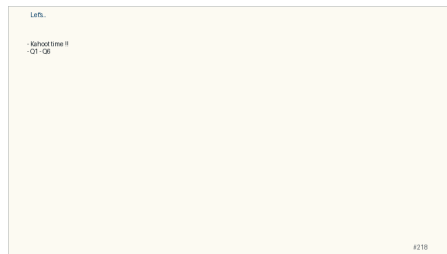
Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 210-218

Pointer to pointer

- The object where a pointer points to, can be also a pointer
- In this case, the type of the object is type `*`, i.e. pointer type
- `int **pp;` means that `pp` is a pointer that points to a pointer that points to integers

- type

- type

- int

- *

- *

- pp

Backup: Pointer to pointer

Original slide 210

- ▶ The object, where a pointer points to, can be also a pointer
- ▶ In this case, the type of the object is type*, i.e. pointer type
- ▶ `int **pp;` means that `pp` is a pointer that points to a pointer that points to integers
- ▶ type
- ▶ type
- ▶ int
- ▶ *
- ▶ pp
- ▶ *

Pointer to pointer

- The object where a pointer points to can be also a pointer
- In this case, the type of the object is type*, i.e. pointer type
- `int **pp` means that `pp` is a pointer that points to a pointer that points to integers

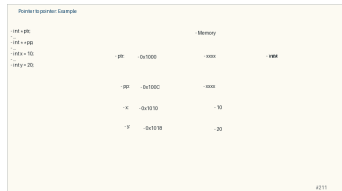
```
graph TD
    type --> type
    type --> int
    pp --> type
```

#210

Backup: Pointer to pointer: Example

Original slide 211

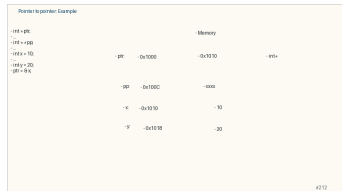
▶ `int *ptr;`
▶ ...
▶ `int **pp;`
▶ ...
▶ `int x = 10;`
▶ ...
▶ `int y = 20;`
▶ Memory
▶ `ptr:`
▶ `xxxx`
▶ `int*`
▶ `int`
▶ `0x1000`
▶ `pp:`
▶ `xxxx`
▶ `0x100C`



Backup: Pointer to pointer: Example

Original slide 212

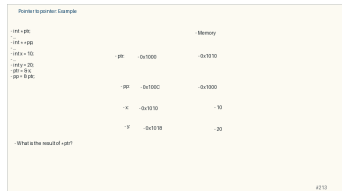
▶ `int *ptr;`
▶ ...
▶ `int **pp;`
▶ ...
▶ `int x = 10;`
▶ ...
▶ `int y = 20;`
▶ `ptr = &x;`
▶ Memory
▶ `ptr:`
▶ `0x1010`
▶ `int*`
▶ `0x1000`
▶ `pp:`
▶ `xxxx`
▶ `0x100C`



Backup: Pointer to pointer: Example

Original slide 213

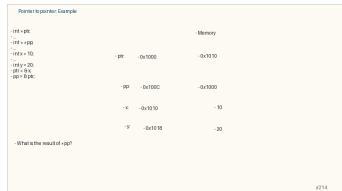
▶ `int *ptr;`
▶ ...
▶ `int **pp;`
▶ ...
▶ `int x = 10;`
▶ ...
▶ `int y = 20;`
▶ `ptr = &x;`
▶ `pp = &ptr;`
▶ Memory
▶ `ptr:`
▶ `0x1010`
▶ `0x1000`
▶ `pp:`
▶ `0x100C`
▶ `0x1000`



Backup: Pointer to pointer: Example

Original slide 214

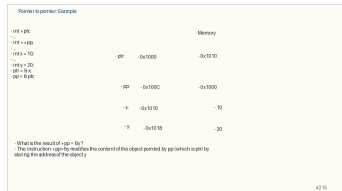
▶ `int *ptr;`
▶ ...
▶ `int **pp;`
▶ ...
▶ `int x = 10;`
▶ ...
▶ `int y = 20;`
▶ `ptr = &x;`
▶ `pp = &ptr;`
▶ Memory
▶ `ptr:`
▶ `0x1010`
▶ `0x1000`
▶ `pp:`
▶ `0x100C`
▶ `0x1000`



Backup: Pointer to pointer: Example

Original slide 215

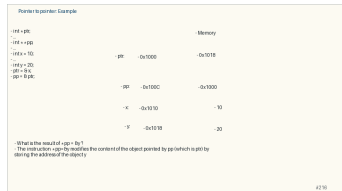
▶ `int *ptr;`
▶ ...
▶ `int **pp;`
▶ ...
▶ `int x = 10;`
▶ ...
▶ `int y = 20;`
▶ `ptr = &x;`
▶ `pp = &ptr;`
▶ Memory
▶ `ptr:`
▶ `0x1010`
▶ `0x1000`
▶ `pp:`
▶ `0x100C`
▶ `0x1000`



Backup: Pointer to pointer: Example

Original slide 216

```
▶ int *ptr;  
▶ ...  
▶ int **pp;  
▶ ...  
▶ int x = 10;  
▶ ...  
▶ int y = 20;  
▶ ptr = &x;  
▶ pp = &ptr;  
▶ Memory  
▶ 0x1018  
▶ ptr:  
▶ 0x1000  
▶ pp:  
▶ 0x100C  
▶ 0x1000
```



Backup: Pointer to pointer: Example

Original slide 217

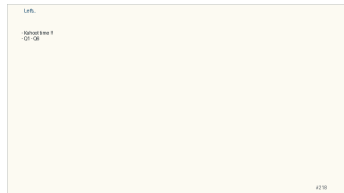
▶ `int *ptr;`
▶ ...
▶ `int **pp;`
▶ ...
▶ `int x = 10;`
▶ ...
▶ `int y = 20;`
▶ `ptr = &x;`
▶ `pp = &ptr;`
▶ Memory
▶ 0x1018
▶ ptr:
▶ 0x1000
▶ pp:
▶ 0x100C
▶ 0x1000

Pointer to pointer: Example		
<pre>- int * ptr; - - int x = 10; - - ptr = &x; - - int y = 20; - pp = &ptr; - pp = 0; ptr;</pre>		Memory
	ptr	0x1000 0x1018
	pp	0x100C 0x1000
	x	0x1010 10
	y	0x1018 20
- What is now the result of *pp?		
42/17		

Backup: Let's...

Original slide 218

- ▶ Kahoot time !!
- ▶ Q1 - Q6



Pointer Arithmetic

Pointer Arithmetic

Live pacing: about 8 min

Question. What does a pointer value name?

Core claim. A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.

Target capability. Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.

Main Path

- ▶ Start from the question: What does a pointer value name?
- ▶ Build the model: A pointer is a typed address; using it safely means separating address, pointed value, and lifetime.
- ▶ End with a checkable action: Students can draw `&x`, `p`, `*p`, pointer-to-pointer, and pointer arithmetic without guessing.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Pointer arithmetic

A pointer is NOT ONLY an address!
It provides information about:
(What type is the object stored in the address where the pointer points to
And thus, what is the size of the object.
Data type's size can be given by the `sizeof()` operator.

#219

Worked Example

- ▶ Use this as the live trace: Trace `int x`, `int *p = &x`, then mutate `x` through both names.
- ▶ Representative source slide: 220
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Pointer arithmetic operators

`- type = ptr; int x;`

Operator Description
`ptr + x` Adds `sizeof(type) * x` to the pointer value
`ptr - x` Subtracts `sizeof(type) * x` from the pointer value
`ptr ++` Increases the pointer value by `sizeof(type)`
`ptr --` Decreases the pointer value by `sizeof(type)`

Example (assuming `sizeof(int) == 4`):

<code>- int * ptr = 0x1000;</code>	<code>- ptr</code>	<code>- 0x1000</code>
	<code>- ptr + 1</code>	<code>- 0x1004</code>
	<code>- ptr + 2</code>	<code>- 0x1008</code>
	<code>- ptr;</code>	<code>- 0x1000</code>

#220

Anecdote Or Motivation

Pointers feel hard because one line can talk about two objects: the pointer variable and the pointee.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Pointer arithmetic: Problems!

You can make a pointer point anywhere, the compiler will NOT complain. The following are valid:

```
ptr = 10000  
ptr = 10000  
char *ptr = (char *) 0x00000000
```

However, problems occur when you attempt to access an invalid memory address by dereferencing the

SECURITY ISSUES

#221

Backup Index

Original slides 219-221

Pointer arithmetic

A pointer is NOT ONLY an address!

It provides information about

What type is the object stored in the address where the pointer points to

And thus, what is the size of the object

Data type's size can be given by the `sizeof()` operator.

Backup: Pointer arithmetic

Original slide 219

A pointer is NOT ONLY an address!

It provides information about

What type is the object stored in the address where

And thus, what is the size of the object.

Data type's size can be given by the `sizeof()` operator

`sizeof(int)`, `sizeof(short)`

The return value depends on the compiler and the target architecture!



Backup: Pointer arithmetic operators

Original slide 220

- ▶ `type *ptr; int x;`
- ▶ Operator Description
- ▶ `ptr + x` Adds `sizeof(type) * x` to the pointer value
- ▶ `ptr - x` Subtracts `sizeof(type) * x` to the pointer value
- ▶ `ptr ++` Increases the pointer value by `sizeof(type)`
- ▶ `ptr --` Decreases the pointer value by `sizeof(type)`
- ▶ Example (assuming `sizeof(int) == 4`).
- ▶ `int *ptr=0x1000;`
- ▶ `ptr`
- ▶ `0x1000`
- ▶ `0x1004`
- ▶ `ptr+1`
- ▶ `ptr+2`
- ▶ `0x1008`
- ▶ `0x1000`
- ▶ `ptr--`

Pointer arithmetic operators

`- type * ptr, int x`

Operator Description
`ptr + x` Adds `sizeof(type) * x` to the pointer value
`ptr - x` Subtracts `sizeof(type) * x` to the pointer value
`ptr ++` Increases the pointer value by `sizeof(type)`
`ptr --` Decreases the pointer value by `sizeof(type)`

Example (assuming `sizeof(int) == 4`)

<code>int *ptr=0x1000;</code>	<code>ptr</code>	<code>0x1000</code>	
	<code>ptr+1</code>	<code>0x1004</code>	
	<code>ptr+2</code>	<code>0x1008</code>	
	<code>ptr</code>	<code>0x1000</code>	

42/50

Backup: Pointer arithmetic: Problems!

Original slide 221

You can make a pointer point anywhere, the compiler

```
ptr + 10000
```

```
ptr - 10000
```

```
char *ptr = (char *) 0x0000ffff
```

However, problems occur when you attempt to access

Usually, the operating system forbids access outside of the program's **alloca**

If the program can access the memory location, the data is not valid

If the program cannot access, it results to segmentation fault

The operating system sends the SEGFault signal to the process that caused th

► SECURITY ISSUES

